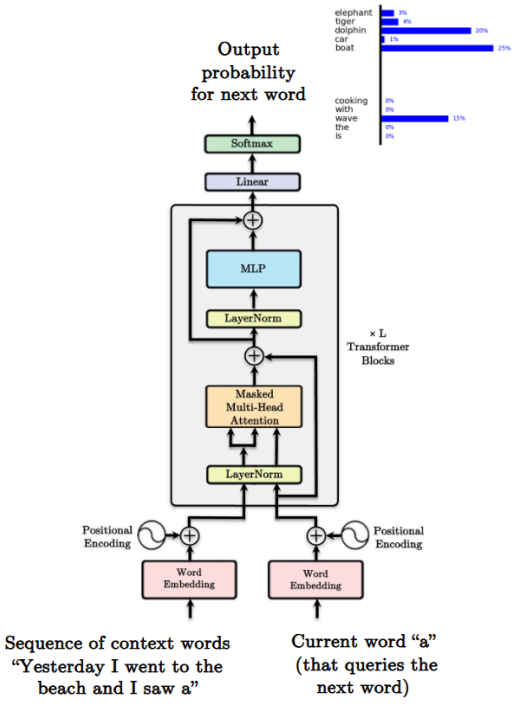
Transformers

1. Motivation [8]:
    1. Different senses, but same embedding: word2vec, GloVe, or other static embeddings assign exactly the same embedding to each of the word 'basin' in 'Amazon basin' vs 'she filled the basin with water' or the word 'bank' in 'river bank' vs 'bank' despite their different senses.
    2. we want to assign different, **contextualized embeddings** based on the surrounding context.
2. Transformers can [2]:
    1. process an unordered set of tokens/vectors
        1. Positional encoding adds spatial information.
    2. self-attention is like clustering and replacing vectors with centers, except
        1. Multiple, complex, learnable similarity functions.
        2. No need to preset number of clusters.
    3. Vectors are iteratively aggregated and transformed.
    4. Vectors can start as purely local information (e.g. 16x16 patch).
3. What kind of word embedding is used in the original transformer?
    1. Recall example of word embeddings such as one-hot, word2vec, GloVe, etc.
    2. There are *two options* for dealing with the Pytorch nn.Embedding weight matrix: [3]
        1. Initialize it with pre-trained embeddings and keep it fixed, in which case it's really just a lookup table.
        2. Initialize it randomly, or with pre-trained embeddings, but keep it trainable. In that case the word representations will get refined and modified throughout training because the weight matrix will get refined and modified throughout training. The transformer uses a random initialization of the weight matrix and refines these weights during training - i.e. it learns its own word embeddings. Note: Recall in backpropagation we can get $\frac{\partial L}{\partial \vec{x}}$.
4. Transformer encoder[6]
    1. The encoder in seq2seq model can replace recurrence through a series of multi-head self-attention blocks.
    2. But this ignores relative position.
        1. A context word one word away is different from one 10 words away.
        2. The attention framework does not take distance into consideration. This can be an issue, for example when, there are multiple duplicated words in a sentence.
    3. Add positional encoding to them to capture the relative distance from one another.
    4. Positional encoding cannot be any random function: A sequence of vectors $P_0, \ldots, P_N$ to encode position
        1. Every vector is unique and uniquely represents time
        2. Relationship between P_t and $P_{t+\tau}$ only depends on the distance between them $P_{t+\tau} = M_\tau P_t$
        3. The linear relationship between P_t and $P_{t+\tau}$ enables the net to lean shift-invariant "gap" dependent relationships
        4. $P_{t+\tau} = M_\tau P_t$
        5. $M_\tau = diag\left(\begin{bmatrix} \cos\omega_l\tau & \sin\omega_l\tau \\ -\sin\omega_l\tau & \cos\omega_l\tau \end{bmatrix}, l = 1, \ldots, d/2\right)$
        6. A vector of sines and cosines of a harmonic series of frequencies
            1. Every $2l$-th component of P_t is $\sin\omega_l t$
            2. Every $2l+1$-th component of P_t is $\cos\omega_l t$
            3. Never repeats
            4. Has the linearity property required
    5. BERT uses the encoder side.
5. Transformer decoder [4], [5]
    1. in NMT the partially decoded sentence is used as query in the first multi-head attention block.
    2. the need for decoder
        1. The decoder is used when the output of the model is a sequence.
        2. If the output of the model is single value, e.g. for a classification task or a single-value regression task, the encoder would suffice.
        3. When predicting multiple values of a time series, a decoder should probably be used that let to condition not only on the input, but also on the partially generated output values.
    3. Decoder = Conditional generator
        1. In math: $P(y_t|x_1, \ldots, x_T, y_1, \ldots, y_{t-1})$

    2. where $x_1, \ldots, x_T$ is the input sequence, $y_1, \ldots, y_{t-1}$ are all the past output items.
    3. Each item in the output sequence must be conditioned on:
        1. The entire input sequence.
        2. All the past output items.
    4. In deep learning the decoder must have access to:
        1. Some kind of encoding of the entire input sequence.
        2. The past states of the decoder.
4. Very similar to the encoder.
5. Masked MHA block to hide future output values during training.
6. The query from the decoder is used with the keys/values from the encoder.
7. Final output probabilities are computed using a projection layer followed by a softmax.
8. GPT only uses the decoder.
9. Decoder self-attention and masked attention:
    1. Decoders can be parallelized during training (only).
    2. Feed the whole output sequence (outputs) in all at once.
    3. Need to ensure model doesn't cheat.
    4. Alter the attention weights to be 0 (set input to softmax to -inf) for all times $t' > t$.
    5. Ensure autoregressive property.
10. Cross-attention
    1. During decoding, the query comes from the outputs, keys and values come from the encoder.
    2. Decoder input "pays" attention to the encoder outputs.
11. Feedforward layers allow for high dimensional computations (nonlinearity).
12. Simply there to allow the model to capture more information.

6. self-attention computed for an T length input requires the computation of $T \times T$ attention weight matrix for each head.
7. Masked self-attention is required for all layers of the decoder
8. We can combine recurrent layers with self-attention layers.
9. Positional encodings are the same in the encoder and decoder, the only thing different is the mask.
10. Transformers are residual machines.
    1. Add & Norm: out = LayerNorm(x + Sublayer(x)),
    2. Sublayer(x) is whatever layer is below the Add & Norm.



References:

1. Formal Algorithms for Transformers, Mary Phuong, Marcus Hutter.
2. CS598, Derek Hoeim, UIUC.

3. The Transformer: Attention Is All You Need
4. Role of decoder in transformer
5. Attention Networks, CMU 11-785 S21, Recitation 9.
6. seq2seq models: Attention models. CMU 11-785, F22 Lecture 17.
7. Xavier Bresson, Lecture 7, Attention Neural Networks.
8. Natural Language Processing, UofU.